DO NOW — RECALL

# Year 13 autumn mock: Paper 2

Name: _____ Class: _____ Date: _____

**1. What makes a program's output testable?**

**2. What is a precondition?**

**3. Which part of a partition is guaranteed to be in its final position?**

**4. Push 4,9,2 onto a stack; pop once; enqueue the popped value to queue [7,8], then dequeue. Give remaining stack and queue.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

Defined success criteria and unambiguous rules.

**2. Model answer**

A requirement that must hold before an algorithm runs.

**3. Model answer**

The pivot (under the specified partition method).

**4. Model answer**

Stack bottom-to-top [4,9]; pop gives 2; queue [7,8,2] then dequeue 7 leaves [8,2].

**Vocabulary — know these terms**

base case — Condition ending recursion.

trace table — Record intermediate algorithm state.

Big O — Asymptotic algorithm growth.

auxiliary space — Extra storage used by an algorithm.

NEW LEARNING

# Read, annotate and explain

### Answer original practice, not OCR past paper

This diagnostic samples programming and algorithms, including learning from earlier Year 12 lessons.

### Time management

Allow roughly 15 minutes per 15 marks.

### Method precision

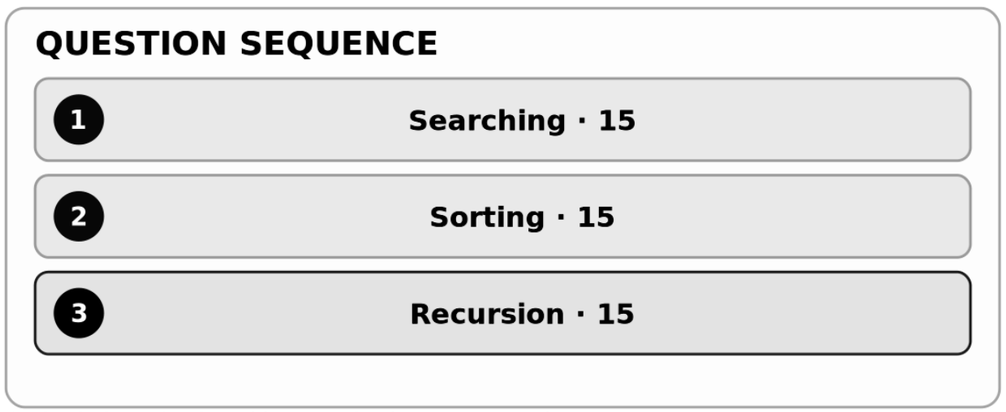
Binary search must shrink bounds, sorting traces must show complete passes, recursion must include a terminating condition.

**Explain one step in your own words, using the correct technical terms.**

## New Learning · original lesson diagrams

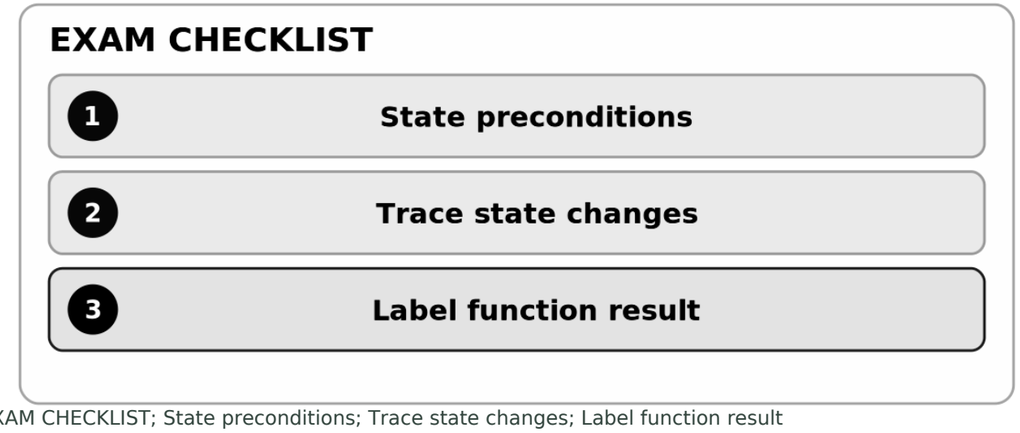
Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




QUESTION SEQUENCE; Searching · 15; Sorting · 15; Recursion · 15

**New Learning**




EXAM CHECKLIST; State preconditions; Trace state changes; Label function result





## WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

**Guided 1 — original CHECK**

Check and explain

Swap 5 and 4. The first full pass ends at [2,4,5].

IMPROVE: Explain one correction to your own reasoning.

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**A correctly written recursive function without a base case is guaranteed to terminate.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

**GREEN PEN — Correct and explain**

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.





I DO — WORKED EXAMPLE

# Follow the worked method

**Unmarked warm-up: trace binary search midpoint for 7 in [1,3,5,7,9].**

**Predict the method**

Check the middle 5 and discard indexes 0–2 because target 7 is larger.

**Check and explain**

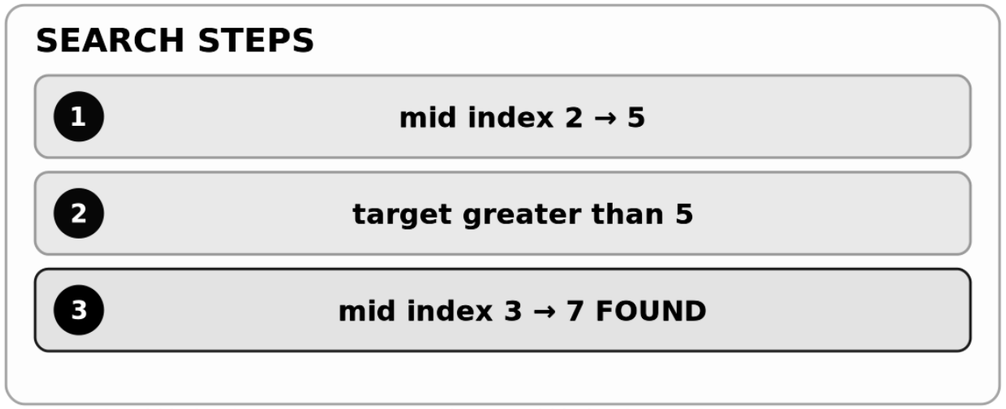
Next middle is index 3 containing 7, so return 3.

## I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Unmarked warm-up: trace binary search midpoint for 7 in [1,3,5,7,9].**

Check the middle 5 and discard indexes 0–2 because target 7 is larger.




SEARCH STEPS; mid index 2 → 5; target greater than 5; mid index 3 → 7 FOUND





YOU DO — INDEPENDENT PRACTICE

## Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

**Question 3**

(a) Write recursive C# function SumTo(n) returning 1+…+n for n≥0, with base case. [8] (b) Trace SumTo(3) showing the recursive calls and returned value. [5] (c) State one drawback of recursive solutions. [2]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

(a) State the key precondition for binary search and explain why it is necessary. [2] (b) Trace binary search for 11 in [1,3,5,7,9,11,13], giving low/high/mid at each step and resulting index. [7] (c) Compare worst-case complexities of linear and binary searches. [6]

### Question 2

(a) Starting with [5,1,4,2], show the array after ascending bubble-sort Pass 1, Pass 2 and a final no-swap checking Pass 3. [9] (b) Compare bubble sort with merge sort in worst-case time complexity and auxiliary space. [6]





WE DO — GUIDED PRACTICE

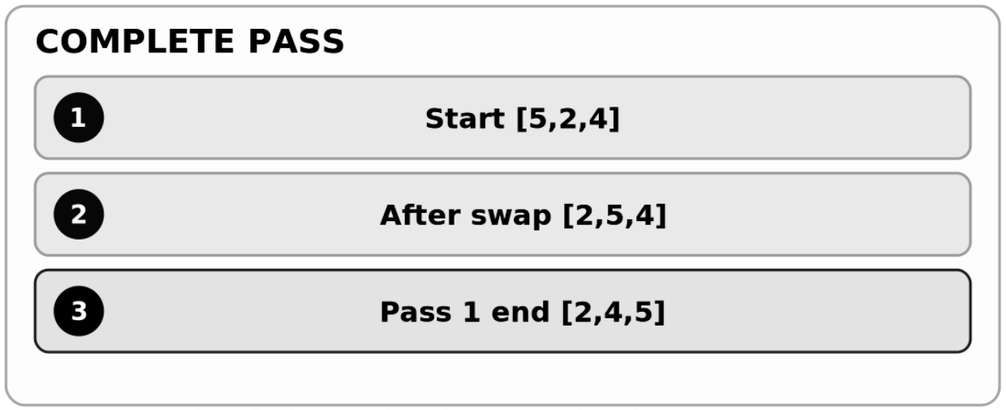
# Try → Discuss → Improve

## We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Unmarked warm-up: trace first bubble pass on [5,2,4].**

Swap 5 and 2 to get [2,5,4].




COMPLETE PASS; Start [5,2,4]; After swap [2,5,4]; Pass 1 end [2,4,5]





## WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

**Guided 1: Predict the method Swap 5 and 2 to get [2,5,4].**

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.